Stoquify migration control-plane remediation prompt

Date: 2026-08-28 Workspace: `E:\ohada saas\Focused projects\stoquify` Prompt type: evidence-led architecture, database-migration, security, controls, and release-assurance implementation brief

Refined Professional Prompt

```
Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Use these permanent core reviewers:

- Principal enterprise/platform architect: preserve domain ownership, dependency order, tenancy boundaries, modularity, and explicit integration contracts.
- Staff backend/domain and integration engineer: protect server-owned business truth, transactional boundaries, APIs, events, idempotency, concurrency, and provider failure handling.
- Principal data/database and migration architect: protect schema integrity, monetary precision, provenance, retention, backfills, rollback safety, and zero-loss migrations.
- Principal application-security, IAM, privacy, and abuse-resistance architect: enforce tenant isolation, RBAC, entitlement, fresh auth, segregation of duties, redaction, secrets safety, auditability, and least privilege.
- Senior frontend and design-systems engineer: identify downstream contract impact; mark user-interface work not applicable unless a real operational surface must change.
- Principal workflow/service designer, accessibility specialist, and localization/content strategist: validate operator journeys and recoverability; mark visual UI work not applicable unless evidence requires it.
- Principal product strategist and business-process analyst: connect migration operations to safe environment onboarding, release ownership, escalation, and measurable outcomes.
- Principal quality engineer and release-assurance lead: require unit, integration, contract, migration, failure-path, rollback, and evidence-producing release gates in proportion to risk.
- Principal SRE/DevSecOps, observability, resilience, and performance engineer: cover deployment order, telemetry, alerting, incident recovery, capacity, latency, and cost.
- Principal SaaS platform, packaging, billing, growth, customer-success, and product-operations strategist: mark commercial packaging not applicable unless migration behavior changes an entitled module or customer rollout contract.

Activate these trust and domain reviewers because their boundaries are material:
```

- Enterprise finance, OHADA accounting, treasury, reconciliation, fraud-risk, and internal-controls specialist.
- Audit, evidence, records-governance, and data-quality specialist.
- Application security, Better Auth persistence, session-assurance, IAM/RBAC, privacy, and tenant-isolation specialist.
- Accounting close, ledger, fiscal-document, reporting, and accountant-portal specialist.
- API, webhook, business-event/outbox, import/export, and third-party boundary specialist.
- Change-management, documentation, training, support, rollout, and operational-readiness specialist.

Project:
Stoquify / AqStoqFlow.

Domain:
PostgreSQL and Prisma migration lifecycle, environment onboarding, immutable migration history, schema drift, destructive-change evidence, and production deployment assurance.

Mission:
Resolve Stoquify's migration-release blocker without resetting any database, rewriting applied migration history, inventing approvals, weakening fail-closed controls, or disturbing unrelated work. Preserve the currently healthy local 79/79 migration history. Replace the brittle global destructive-risk gate with a coherent migration control plane that:

1. proves full repository/database history integrity for the selected target;
2. computes the exact target-specific pending migration set;
3. applies destructive-risk review only to pending migrations while retaining immutable audit visibility over the full catalog;
4. distinguishes a genuinely empty-database execution path from an existing-database baseline-adoption path;
5. never runs the baseline-only bridge on an existing populated database;
6. binds approvals and evidence through one documented canonical hashing contract;
7. performs post-deploy history and schema verification; and
8. produces current, immutable, release-scoped evidence.

Current evidence baseline to verify, not blindly trust:

- The repository contains 79 non-empty migrations.
- The configured local database currently reports 79/79 completed migrations, zero unfinished or rolled-back rows, zero unknown or duplicate successes, and zero checksum mismatches.
- `prisma/migrations/20260611130000_accounting_auth_baseline_bridge/migration.sql` contains 13 destructive findings: 12 dropped columns and one dropped table.
- That migration explicitly says it is baseline-only. Existing databases with application data or the target auth/accounting schema must be verified and then adopted with `prisma migrate resolve --applied`; they must not execute it.
- `prisma/migration-risk-approvals.json` contains zero approvals.
- The live destructive-evidence bundle is rejected: 0/14 required artifacts are complete, the candidate was captured in rolling/dirty mode, and bound mutable files have drifted.

- The most recent fresh replay certificate covers 62 migrations, while a later schema/seed reconciliation covers 64; neither certifies the current 79-migration catalog.
- Two migration directories share the timestamp prefix `20260818120000`. Their full migration names are unique and deterministically ordered, but timestamp-only tooling can collide.
- The destructive-risk gate canonicalizes line endings before hashing, while the older maker-checker packet binds the raw file bytes. Preserve both existing identities in evidence, but define one canonical approval identity and a verified mapping during the transition.
- Current dependency versions are Prisma CLI/client 6.19.3 and `@prisma/adapter-pg` 7.9.1. The adapter is not imported by application code. Treat this as dependency hygiene, not as the proven cause of migration drift.

Required tasks:

1. Reconfirm current truth.
   - Inspect `prisma/schema.prisma`, `prisma/migrations/`, both approval registries, the migration gates, the production runbook, package scripts, recent migration reports, and focused tests.
   - Read the configured target only after classifying it without exposing credentials. Never print or retain a database URL, migration error log, token, or secret.
   - Inventory every controlled environment by redacted environment ID, target class, database owner, migration count, current baseline state, backup/PITR status, and selected path.

2. Freeze migration history safely.
   - Do not edit, rename, reorder, delete, or squash a migration that has been applied anywhere.
   - Introduce a deterministic catalog manifest or equivalent CI check that records migration name plus accepted raw/LF/CRLF hashes and fails on unauthorized mutation.
   - Grandfather the two existing duplicate timestamp prefixes by exact full name, and fail CI on any new duplicate 14-digit timestamp prefix.

3. Separate the gates.
   - Full-history integrity gate: block unknown migrations, checksum divergence, unfinished rows, duplicate successful rows, invalid approvals, and repository catalog mutation.
   - Target-pending risk gate: after the full-history gate passes, derive pending migrations from the selected target and require exact-hash destructive approval only for that pending set.
   - Bootstrap gate: on a new isolated database, replay the full catalog, prove the baseline tables are empty at the destructive boundary, run twice to prove no-op idempotency, and diff the resulting schema against `prisma/schema.prisma`.
   - Existing-database adoption gate: never auto-run `migrate resolve`. Require a redacted census, schema/data compatibility proof, backup and restore rehearsal, named maker/checker, and a controlled manual action.
   - Post-deploy gate: rerun history health, `prisma migrate status`, schema drift classification, and focused auth/accounting smoke checks before declaring the release complete.

4. Repair evidence semantics.
   - Define one canonical migration identity for approvals. Prefer normalized LF UTF-8 SQL plus an explicit raw-byte hash for transport integrity.
   - Version the hash contract and include the canonicalization algorithm in every packet.
   - Bind an approval to immutable migration SQL, destructive-operation inventory, target-path classification, rehearsal bundle, maker/checker decision, and optional expiry/revocation.

   - Do not make historical approval validity depend on the current mutable `package.json`, live Git HEAD, current Prisma schema, or application consumer files. Snapshot those as release evidence instead.
   - Bind each deployment to a release manifest that references the migration approval and current source commit without invalidating the historical approval when unrelated source files later change.

5. Close the baseline decision once.
   - For an existing database whose schema/data already match the post-baseline state, use the controlled resolve-only path after evidence and independent approval. Prove application tables are unchanged and only migration metadata changes.
   - For a genuinely empty database, use the execute path only after a current 79/79 blank replay, zero-row destructive-boundary proof, transaction failure injection, auth regression, and restore rehearsal.
   - If any environment has legacy auth/accounting data that does not match either path, stop. Design additive expand/backfill/verify/contract migrations; never force the baseline bridge through that data.

6. Align release automation.
   - Order release checks as: secret/target preflight -> full-history health -> pending-risk gate -> backup/PITR attestation -> deploy -> post-deploy history -> drift classification -> auth/accounting smoke -> evidence archive.
   - Keep local and preview database mutation disabled by default.
   - Require explicit isolated preview/staging target attestation when deployment is opted in.
   - Preserve fail-closed behavior and safe error redaction.

7. Verify and report.
   - Add focused tests for history/pending-set separation, baseline path selection, hash compatibility, new timestamp collision rejection, stale/revoked approval rejection, execute/no-op replay, resolve-only metadata behavior, and post-deploy verification.
   - Save the implementation report and machine-readable evidence under a governed migration evidence directory.
   - Report passed, failed, skipped, timed-out, and externally blocked checks separately.

Execution checklist:

- [ ] Confirm repository catalog count, naming, non-empty SQL, and accepted checksums.
- [ ] Confirm current target classification without leaking the URL.
- [ ] Query `_prisma_migrations` read-only and establish the exact pending set.
- [ ] Confirm no migration files are dirty before changing control-plane code.
- [ ] Add or adjust only the migration scripts, focused tests, package commands, runbook, and evidence schemas needed for this mission.
- [ ] Preserve unrelated dirty-worktree changes and do not touch unrelated lint warnings.
- [ ] Run the smallest focused checks first.
- [ ] Run a current 79/79 disposable replay only against an explicitly verified local/isolated database.
- [ ] Do not access or mutate production without explicit authority and target evidence.
- [ ] Do not author human approvals on behalf of a reviewer.

Evidence to inspect:

- `prisma/schema.prisma`
- `prisma/migrations/`
- `prisma/migrations/20260611130000_accounting_auth_baseline_bridge/migration.sql`
- `prisma/migration-risk-approvals.json`
- `prisma/migration-history-checksum-approvals.json`
- `scripts/prisma-production-migration-gate.js`
- `scripts/prisma-migration-history-health-check.js`
- `scripts/prisma-fresh-replay-certification.js`
- `scripts/prisma-local-fresh-bootstrap.js`
- `scripts/prisma-destructive-migration-evidence-gate.js`
- `scripts/prisma-destructive-migration-binding-refresh.js`
- focused tests under `scripts/__tests__/prisma-*.test.js`
- `docs/operations/runbooks/prisma-production-migrations.md`
- `docs/reconcile-destructive-migration/`
- `docs/blockers/stoquify-migration-risk-maker-checker-packet-2026-08-13.json`
- `what-next/prisma-migration-deployment-readiness.{md,json}`
- `what-next/prisma-migration-history-health*.{md,json}`
- `what-next/prisma-fresh-replay-certification-referral.{md,json}`
- `what-next/database-schema-migration-seed-reconciliation-2026-08-15.{md,json}`
- `graphify-out/GRAPH_REPORT_app.md` and relevant finance/security communities for downstream impact only.

Expected artifacts:

- A migration-control-plane decision record describing full-history and pending-risk boundaries.
- Versioned migration catalog/hash manifest and schema.
- Environment census schema and redacted per-environment records.
- Current 79/79 blank-replay report and JSON evidence.
- Existing-database resolve-only rehearsal evidence where that path applies.
- Backup/restore and failure-injection evidence.
- Focused gate tests.
- Updated production migration runbook.
- Release-scoped immutable evidence manifest.
- A concise implementation report with unresolved external actions and named owners.

Focused verification commands:

```powershell
npm run prisma:migration:safety:gate
npm run prisma:migration:history:health
npm run prisma:migrate:status
npm run prisma:fresh-replay:certify
npm run prisma:migration:evidence:gate
npm run prisma:validate
npm test -- --runInBand scripts/__tests__/prisma-production-migration-gate.test.js
```

```
scripts/__tests__/prisma-migration-history-health-check.test.js scripts/__tests__/prisma-local-fresh-bootstrap.test.js scripts/__tests__/prisma-destructive-migration-evidence-gate.test.js
```

Use database commands only after the target is classified and explicitly verified as local or isolated. Use the repository's bundled commands rather than improvising direct production mutations.

Risk controls:

- Zero destructive resets, drops, or history rewrites.
- No production URL copied into local configuration.
- No automatic `prisma migrate resolve`.
- No approval generated or signed by automation.
- No secret or database URL in reports.
- Existing database data, auth/session provenance, ledger evidence, tenant boundaries, and audit history must remain unchanged unless an explicitly approved additive migration says otherwise.
- Prefer fix-forward migrations after an execution failure; use resolve only after independent state verification.
- Keep old application and schema compatibility through expand/backfill/verify/contract sequencing for any future destructive change.
- Preserve the user's unrelated modified and untracked files.

Success criteria:

1. Current repository catalog and each controlled target have a redacted, reproducible history-health record.
2. A healthy target with no pending migrations is not blocked by historical destructive SQL whose applied checksum matches repository truth.
3. A target with a pending destructive migration remains blocked without a valid exact-hash, evidence-bound human approval.
4. An existing populated target can never execute the baseline-only bridge through ordinary deployment automation.
5. A current isolated blank target replays all 79 migrations, a second deploy is a no-op, and unexpected schema drift is zero.
6. Raw-byte and canonical migration hashes are both reproducible and their roles are unambiguous.
7. New duplicate timestamp prefixes, applied-file mutation, unknown history, checksum drift, unfinished rows, stale approvals, and revoked approvals fail closed.
8. Post-deploy verification proves history, schema, auth/session, and accounting controls before release completion.
9. No database, migration approval, production target, or certification claim is changed without the required human authority and evidence.

Non-goals:

- Do not reset or reseed an existing database.
- Do not rewrite, rename, squash, or delete applied migrations in this phase.
- Do not approve the 13 destructive findings automatically.
- Do not treat current local 79/79 health as production evidence.
- Do not refactor unrelated application modules, UI tables, lint warnings, or business workflows.

- Do not add a migration merely to make reports look clean.
- Do not claim OHADA, accounting, security, privacy, accessibility, or production certification.

```
## Optional Next Prompts

1. **Control-plane implementation:** Implement only the full-history/pending-risk gate separation, canonical hash contract, timestamp-collision ratchet, and focused tests. Do not touch a database.
2. **Current blank replay:** Against a newly created, explicitly verified local PostgreSQL database, certify a 79/79 replay, second-run no-op, schema drift, failure injection, and auth/accounting smoke evidence.
3. **Environment census:** Produce the redacted environment census and classify each database as `EXECUTE_EMPTY`, `RESOLVE_EXISTING`, or `REQUIRES_ADDITIVE_REMEDIATION`; do not mutate any target.
4. **Baseline maker packet:** Rebuild the migration-risk evidence bundle from a clean candidate with current hashes and completed technical evidence, then stop for an independent human checker.
```